



RIPHAH
INTERNATIONAL UNIVERSITY

SYED HASSAN RAZA

65700

GAME PROGRAMMING

ASSIGNMENT_3

Apex Legends – Code Architecture Design

1. Game Overview

Apex Legends is a **team-based Battle Royale shooter** where squads of three players compete against others on a large map. Core mechanics include:

- Fast-paced combat with unique hero abilities.
- Looting weapons, armor, and items.
- Dynamic shrinking play zone.
- Squad-based teamwork and respawn mechanics.

2. High-Level Architecture

Major Systems

- **Player System** Handles player profiles, stats, abilities, and progression.
- **Matchmaking System** Matches squads based on skill, region, and latency.
- **Combat System** Governs shooting, damage calculation, abilities, and hit detection.
- **Inventory System** Manages weapons, ammo, healing items, and equipment.

- **Game State Management** Controls match lifecycle: lobby → in-game → results.
- **UI System** Provides HUD, menus, inventory screens, and notifications.

3. Code Structure

Organization

- **Core Classes**
 - PlayerManager – Manages player data, abilities, and respawn.
 - Matchmaker – Handles lobby creation and squad assignment.
 - CombatManager – Processes weapon fire, ability effects, and damage.
 - InventoryManager – Tracks items, pickups, and equipment.
 - GameStateController – Controls transitions between game states.
 - UIController – Updates HUD, menus, and player feedback.

Components

- **Networking Layer**
 - ServerManager – Synchronizes state across clients.
 - ClientHandler – Manages player inputs and updates.
- **Physics & Rendering**
 - PhysicsEngine – Collision detection, bullet trajectories.
 - RenderEngine – Visual effects, animations, map rendering.

Communication Between Systems

- **Event Bus / Observer Pattern**
 - Example: Combat events trigger UI updates (e.g., health bar).
 - Matchmaking events notify GameStateController to start a match.

4. Design Decisions

- **Modular Architecture:** Each system is independent, making it easier to scale and maintain.
- **Event-Driven Communication:** Reduces coupling between systems, allowing flexibility for future updates.

- **Client-Server Model:** Ensures fairness and prevents cheating by keeping authoritative logic on the server.
- **Component-Based Design:** Players, weapons, and abilities are built from reusable components, supporting rapid iteration.